Software Requirements Specifications

Lost Grenadiers

GitErDone

Audrey Dickson, Kaden Langford, Andrew Swartz

Indiana University Southeast

CSCI P445: CAPSTONE PROJECT DESIGN

Dr. Ron Finkbine

October 2, 2025

Individual Contributions

Audrey Dickson
- Project management
- UI/UX design
- UI implementation

Kaden Langford
- Database schema design
- DevOps

Andrew Swartz
- Security Architect
- Backend Engineer
- Penetration Testing and Server Hardening

Key Personnel

Project Manager: Audrey Dickson

Lead Developer: Kaden Langford

UI/UX Designer: Audrey Dickson

QA/Tester: Andrew Swartz

I. Introduction

a. Purpose: The purpose of Lost Grenadiers is to provide the students of Indiana University Southeast with an intuitive campus navigation tool. The system allows users to locate their classrooms and buildings using real-time global positioning, turn-by-turn routing, and indoor & outdoor mapping. This directly addresses common student endeavors of campus navigation and reduces reliance on staff assistance or printed maps.

b. Definitions:

i. GPS: The Global Positioning System, a collection of satellites that publicly provide information about the current time and their current location.

ii. UI: The term "UI" refers to the user interface of software in which the person operating it will communicate with the computer through.

iii. API: Application Programming Interface refers to the set of rules or protocols that allow two (or more) different software systems to communicate.

iv. GIS: A Geographic Information System is a computer-based structure or program that collects, maintains, analyzes, and displays geographical information.

v. PII: Personally identifiable information any type of data that can be used to distinguish an individual's identity.

vi. MTTRS: Mean time to restore service refers to the average amount of downtime required to restore the system to its original functionality and state after a failure.

vii. DMZ: This refers to a demilitarized zone, which in networking is an isolated part of a network that is logically placed beyond the internal network of the devices hosting it.

c. Background

i. Indiana University Southeast is a medium-sized regional campus where students frequently struggle with navigation. A notable case is first-year freshman students. They often face difficulties locating classrooms, offices, and services, especially during their first semester. While physical maps and campus websites currently exist, these resources are static and lack real-time navigation as well as indoor guidance. They are heavily limited in features.

d. System Overview

i. Lost Grenadiers is a mobile-friendly website accessible to everyone on campus. It combines geolocation information, orientation information, and APIs to track the location of the student requesting the service and rapidly reply with the optimal route to reach a desired destination.

e. References

i. Leaflet. (n.d.). *Leaflet API reference* (Version 1.9.4). Leaflet. https://leafletjs.com/reference.html

ii. Pallets Projects. (n.d.). *Welcome to Flask – Flask Documentation (3.1.x).* Flask. https://flask.palletsprojects.com/en/stable/

iii. Mozilla. (2025, May 28). *Geolocation API – Web APIs.* MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API

iv. Mozilla. (2025, September 18). *Detecting device orientation – Web APIs.* MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Device_orientation_events/Detecting_device_orientation

v. The PostgreSQL Global Development Group. (n.d.). *About PostgreSQL.* PostgreSQl. https://www.postgresql.org/about/

vi. TigerData. (2024, March 6). *PostgreSQL extensions: pgRouting.* TigerData Learn. https://www.tigerdata.com/learn/postgresql-extensions-pgrouting

vii. Cisco Systems. (n.d.). *Configure Inter-VLAN routing with Catalyst switches.* Cisco Support. https://www.cisco.com/c/en/us/support/docs/lan-switching/inter-vlan-routing/41260-189.html

II. Overall Description
   a. Product Perspective
      i. The Lost Grenadiers website prompts a user for their desired classroom name. It then replies to our system the datapoints of the location of the student and the desired location of the student. Within our backend database containing a graph of all walkable paths and all possible classrooms, the optimal route is calculated via a path finding algorithm.
   b. System Interfaces
      i. The system interface is the generic Web browser. Supported web browsers include Chrome, Firefox, Safari, and Microsoft Edge.
   c. User Interfaces
      i. Mobile-first design with responsive layout.
      ii. Interactive map: movable, draggable, orientable. It also has pinch-to-zoom and panning gestures.
      iii. Search bar for buildings and classrooms.
      iv. Routing instructions displayed as text or displayed as arrows.
   d. Hardware Interfaces
      i. Smartphone GPS receiver.
      ii. Internet connection.
   e. Peripherals
      i. Built-in smartphone sensors such as the GPS reader, the accelerometer, the gyroscope, and the magnetometer will be required and utilized by our service.
   f. Software Interfaces
      i. Web Application hosting: Flask
      ii. SSL Encryption: Let's Encrypt
      iii. Database: PostgreSQL
      iv. Mapping: Leaflet
   g. Communication Interfaces
      i. HTTPS connection between client and server.
      ii. Browser APIs for geolocation data.
   h. Memory Constraints
      i. This setup allows for minimal client storage requirements. All modern devices are created with a factory default web browser installed.
      ii. This setup forces all storage onto the server side, which is intentional. This means we keep track of all data points (edges and nodes on the campus graph) and no routes are saved on the client machines. It ensures inaccurate data is not cached in their browser and forces all route requests through our server.
   i. Product Functions
      i. Display interactive campus map with real-time positioning through HTML documents.
      ii. Provide timely route calculations from current location to building / classroom.
      iii. Allow search of room number or building name.
      iv. Offer indoor navigation for supported buildings.
   j. User Characteristics
      i. All users are familiar enough with smart phones and following link to our site.
         1. Primary users: Students of IUS.

2. Secondary Users: Faculty, staff, visitors, vendors, campus police, emergency services.

k. Performance Requirements
   i. Map interactions must respond within 10 seconds under normal network conditions.
   ii. GPS updates must be refreshed at least once every three seconds.
   iii. The system should support at least 300 concurrent users without any performance degradation.
   iv. Indoor navigation should return results within 20 seconds pending GPS reachability.

III. Design Constraints

a. Operations
   i. System operates continuously as long as server host remains online.
   ii. Updates are pushed upon building layout modifications or campus construction.\

b. Site Adaptation Requirements
   i. Hosting server with SSL certificate.
   ii. Database update process for staff to maintain map accuracy.

c. Constraints
   i. GPS accuracy may vary depending on device location and its received signal.
   ii. An internet connection is required for our site to be usable.
   iii. Privacy concerns for location data do exist.
   iv. A backend server will be required to host the website. It will need at least 4 cores and at least 8GB of memory.
   v. A backend L3 switch will be required to isolate the server hosting the website onto a DMZ section of a network to protect the home devices.

d. Assumptions
   i. All users have a method of internet access through Wi-Fi and/or cellular data.
   ii. All users have a physical device, ideally smart phones, for contacting our website.

e. Logical Database Requirement
   i. The database is built on PostgreSQL with the PostGIS extension to handle geospatial data. PostGIS adds support for storing, indexing, and querying spatial objects- points, lines, and polygons. The high-level schema is:
      1. Building table: Stores the building code (e.g. LF = Life Sciences, PS = Physical Sciences), the building name, and geometry of the building such as the entrances.
      2. Room table: Stores each room code broken into its components. Room id (e.g. LF119) contains all necessary high-level data because of the name, the floor, and the room number. This table also includes the geometry of the room / connection path to hallway.
      3. Edge table: Stores the network of connections between locations/buildings both indoors and outdoors.
      4. Node table: Stores the listing of points used in pathfinding by pgrouting.
   ii. PostGIS allows for the easy collection and import of these data fields by layering an overhead geographical map- one obtained from OSM- beneath a drawable graph layer for the custom building of the graph nodes. It converts the data points from the drawn graph into their corresponding (x,y) coordinates on a map.
   iii. PostGIS also allows for simple modification of the data within the table in the event of construction, building layout changes, etc.

f. Dependencies
   i. Reliance of many different applications, services, or processes:

1. Leaflet: Leaflet.js is an open-source JavaScript library for displaying interactive maps in a web browser as well as displaying points of interest.
2. Geolocation API: A web API built into most browsers that allows for the estimation of user location via Wi-Fi, GPS, IP, Cell Towers, etc.
3. Flask: A lightweight open-source web framework for Python that allows for simple API and endpoint creation in the backend.
4. PostgreSQL: An open-source object relational database system that extends the SQL language and is used for maintaining graph data.
5. Secondary Geolocation API: OpenStreetMap is an online, free, and open-source map of the world. Its API is primarily used in obtaining our data points for nodes on the graph within our database.
6. Proxmox: An open-source operating system that is essentially a hypervisor itself without any excess load or bloatware. It will allow us to virtualize/containerize our backend and frontend.
7. QGIS: An open-source desktop GIS that allows users to create, edit, visualize, analyze, and publish geospatial information.
8. Pgrouting: An open-source extension to PostgreSQL written in C++ that provides routing functionality and graph algorithms (e.g. Dijkstra, A*) for network analysis.

ii. Regularly reviewed and updated graphs for room and building data.

IV. Software System Attributes

a. Reliability

i. Reliability primarily depends upon the calculation of the correct data path. An accurate method we will use to calculate overall reliability as the project progresses will be by percentage of accurate mapping data. An accurate map ensures the students ability to reach their destination and thus results in a higher score for review.

ii. Target: If at least 99% of user requests display the correct route successfully, we can consider the project fully successful.

b. Availability

i. Availability primarily depends upon the activity and uptime of the server when requested. We cannot measure statistics on any failed connection attempts; we can only measure our statistics on successful connections to the server. This means we must choose a different method of measuring availability, which will be the overall uptime of the server. A higher uptime means more users can access it throughout the day and thus results in a higher score for review.

ii. Target: If at least 99.9% of the first week of a given semester the server is up, and at least 98% of the remaining days in the semester the server is up, we can consider the project fully successful.

c. Security

i. Security primarily depends upon the protection of:

1. The home devices of the person hosting the server (Andrew Swartz).
2. The personal data, geographical data, and devices of the students who have connected to the server.

ii. A secure system that also lacks PII collection will entice more users to utilize the website. A system riddled with viruses would cause less people to use our site, therefore passing a security penetration test will result in a higher score for review.

iii. Guaranteed target: Absolutely zero PII will be stored from anyone unwilling to share it.

iv. Target: The site can withstand a penetration test from attacks such as SQL injection.

d. Maintainability
    i. Maintainability primarily depends upon how quickly the system can be restored after a failure. This can be implemented through:
        1. Version control systems such as GIT
        2. Database backup systems
    ii. The lower the MTTRS, the less downtime our system would have, thus resulting in a higher score for review.
    iii. Target: The goal for an MTTRS will be 30 minutes for 95% of incidents.

e. Portability
    i. Portability primarily depends upon the ease of use and the consistency of use across different environments or platforms. The more diverse platforms it can be available to will result in a larger number of users, meaning a vast platform compatibility will result in a higher score for review.
    ii. Target: The goal for is the website will be accessible through Chrome, Firefox, Safari, and Microsoft Edge, across devices such as mobile phones (iOS and Android) and personal computers with different operating systems (Microsoft Windows, macOS, and Linux).